

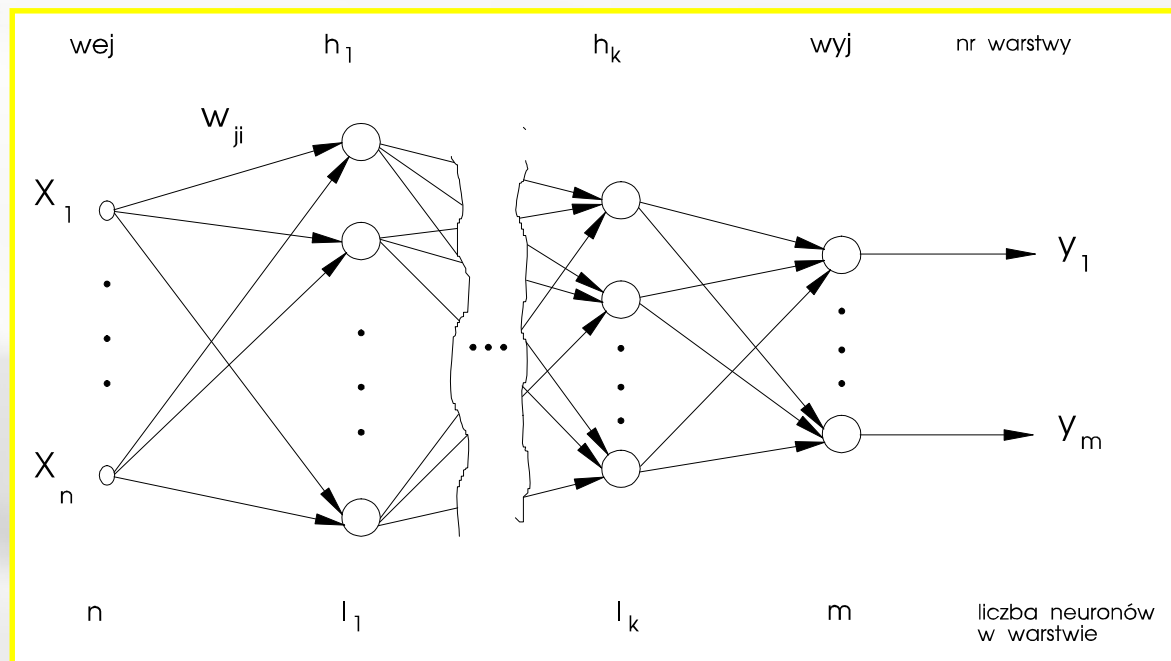
Sieci jednokierunkowe

Architektury sieci neuronowych

- ✓ Rzeczywista moc obliczeń neuronowych wynika z połączenia wielu neuronów w sieci tworzące różnorodne struktury
- ✓ Podział SSN ze względu na architekturę
 - Sieci jednokierunkowe
 - jednowarstwowe
 - wielowarstwowe
 - Sieci rekurencyjne
 - Sieci komórkowe

Sieci jednokierunkowe (feedforward)

- ✓ Charakteryzują się:
 - **warstwowym ułożeniem neuronów** (warstwy mogą zawierać różne ilości neuronów),
 - **połączeniami jedynie pomiędzy neuronami z sąsiednich warstw** (Połączenia pomiędzy warstwami są tak zaprojektowane, że każdy element warstwy poprzedniej jest połączony z każdym elementem warstwy następnej),
 - **jednokierunkowym przepływem informacji**.
- ✓ Zadaniem neuronów z warstwy wejściowej jest wstępna obróbka analizowanego sygnału (np.: normalizacja, kodowanie itp.).
- ✓ Neurony z warstw ukrytych oraz warstwy wyjściowej odpowiedzialne są za przetwarzanie decyzyjne.
- ✓ Sieć udziela odpowiedzi na wyjściach neuronów warstwy ostatniej.



✓ Budowa sieci:

- warstwa wejściowa,
- k warstw zwanych ukrytymi
- warstwa wyjściowa.
- sieć posiada n wejść oraz m wyjść.

✓ Wielkości w_{ji} reprezentują wagi poszczególnych połączeń.

- indeks pierwszy (j) oznacza numer neuronu w warstwie odbierającej sygnał,
- indeks drugi (i) numer neuronu w warstwie emitującej sygnał (lub numer wejścia neuronu w warstwy docelowej).

- ✓ Jeśli zdefiniujemy macierze:

$$\mathbf{y}^{wyj} = [y_1^{wyj}, y_2^{wyj} \dots y_m^{wyj}]^T;$$

$$\mathbf{x}^{wyj} = [x_1^{wyj}, x_2^{wyj} \dots x_{l_k}^{wyj}]^T;$$

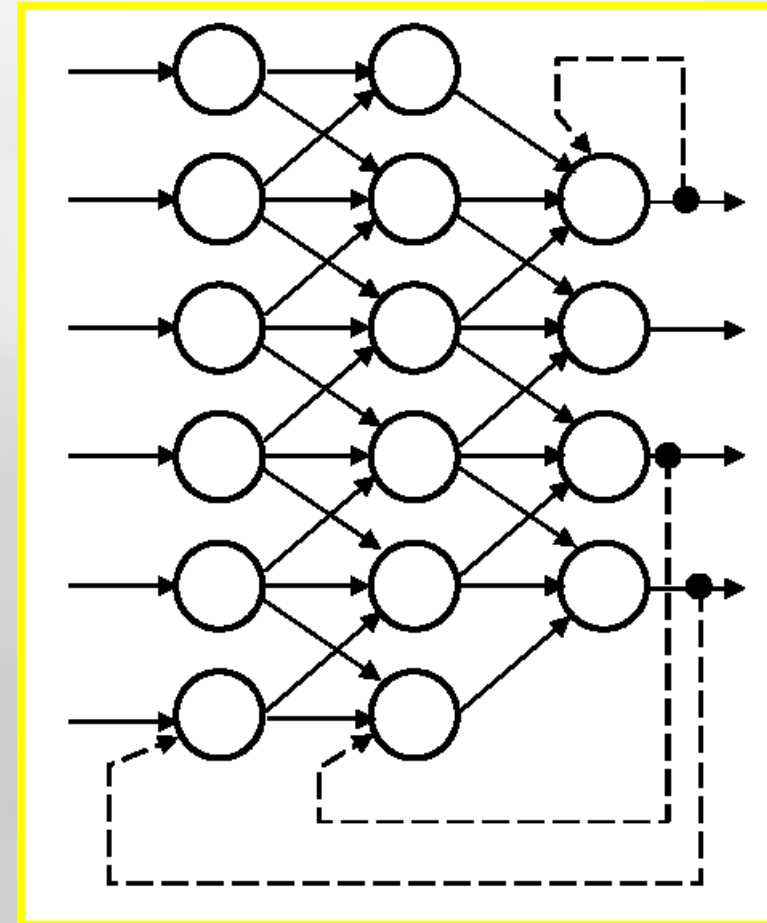
$$\mathbf{w}^{wyj} = \begin{bmatrix} w_{1,1}^{wyj} & w_{2,1}^{wyj} & \dots & w_{m,1}^{wyj} \\ w_{1,2}^{wyj} & w_{2,2}^{wyj} & \dots & w_{m,2}^{wyj} \\ \vdots & & & \vdots \\ w_{1,l_k}^{wyj} & w_{2,l_k}^{wyj} & \dots & w_{m,l_k}^{wyj} \end{bmatrix}$$

to wyjście sieci opisuje zależność (nie uwzględnia BIAS)

$$\mathbf{y}^{wyj} = \mathbf{f}^{wyj} \left(\mathbf{w}^{wyjT} \mathbf{f}^{h_k} \left(\mathbf{w}^{h_kT} \mathbf{f}^{h_{k-1}} \left(\dots \mathbf{f}^{h_1} \left(\mathbf{w}^{h_1T} \mathbf{x}^{wej} \right) \right) \right) \right)$$

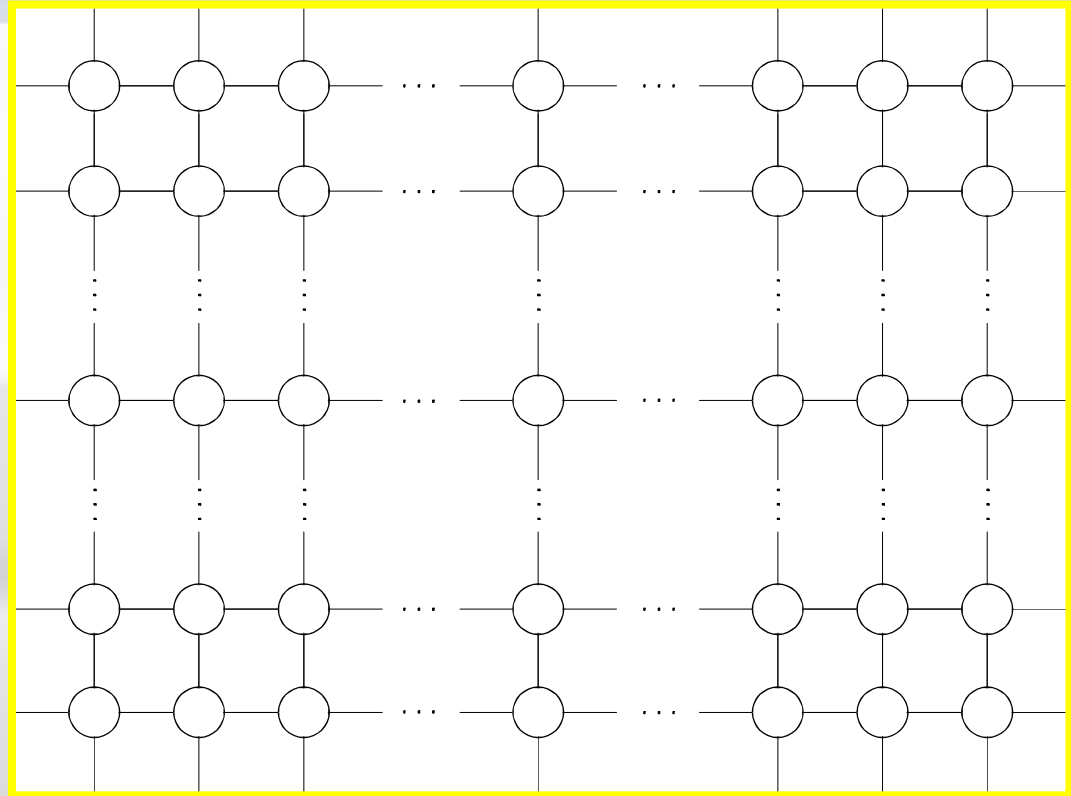
Sieci rekurencyjne

- ✓ Charakteryzują się:
 - dwukierunkowym przepływem informacji,
 - występowaniem w ich pracy przebiegi dynamiczne
- ✓ Do najczęściej spotykanych sieci rekurencyjnych należą
 - sieci Hopfielda,
 - maszyna Boltzmana,
 - sieci BAM (Bidirectional Associative Memory),
 - sieci ART (Adaptive Resonance Theory).

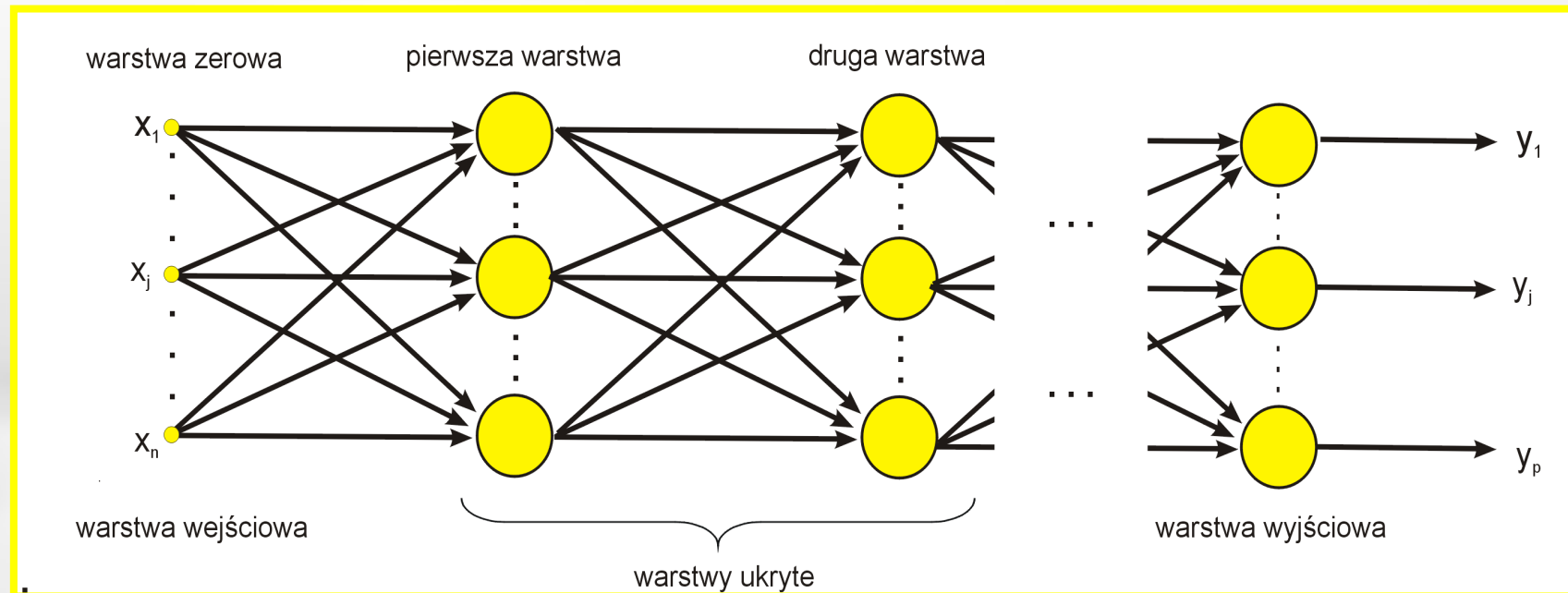


Sieci komórkowe

- ✓ Mają najczęściej strukturę płaskiej prostokątnej siatki geometrycznej.
- ✓ Każda komórka połączona jest ze wszystkimi neuronami znajdującymi się w jej sąsiedztwie wg. schematu jednakowego dla całej sieci.
- ✓ Wszystkie neurony w sieci komórkowej posiadają jednakowe funkcje aktywacji.



Ogólna charakterystyka sieci jednokierunkowych



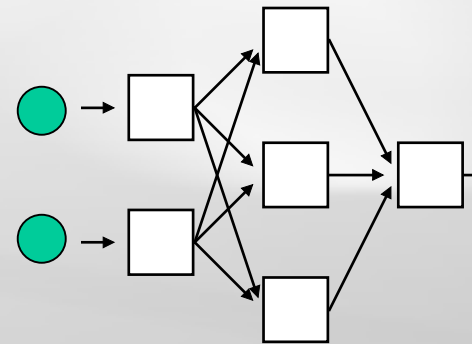
- ✓ W sieciach jednokierunkowych można wyróżnić uporządkowane warstwy (w tym warstwę wejściową i warstwę wyjściową).
- ✓ Połączone są tylko neurony warstw sąsiednich, wg zasady „każdy z każdym”.
- ✓ Połączenia pomiędzy warstwami są asymetryczne.
- ✓ Sygnały przesyłane są od warstwy wejściowej poprzez warstwy ukryte (jeśli występują) do warstwy wyjściowej (w jednym kierunku).
- ✓ Neurony warstwy wejściowej posiadają tylko jedno wejście i uproszczoną funkcję przejścia (umownie jest to warstwa zerowa sieci).

Ważniejsze metody uczenia sieci jednokierunkowych wielowarstwowych

- ✓ *Back Propagation* (wstecznej propagacji błędów),
- ✓ *Quick Propagation* (szybkiej propagacji błędów).
- ✓ *Conjugate Gradients* (gradientów sprzężonych),
- ✓ *Quasi-Newton* (metoda zmiennej metryki),
- ✓ *Levenberg – Marquardt* (metoda paraboloidalnych modeli funkcji błędów),
- ✓ Inne metody „nieklasyczne” (angażujące zmienne metryki, metody bezgradientowe itp.)

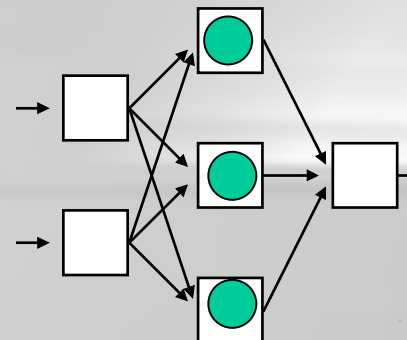
Etapy działania sieci neuronowej

- wprowadzenie na wejście sieci p -tego wektora wejściowego ($\mathbf{x}_p$),



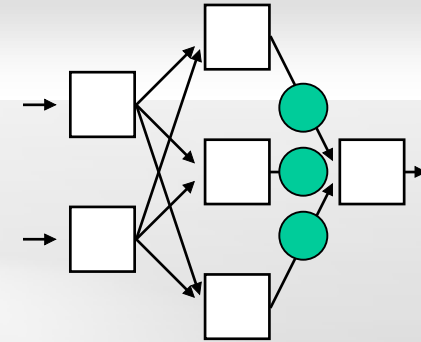
- obliczenie pobudzenia u_{pj}^h neuronów warstwy ukrytej zgodnie ze wzorem:

$$u_{pj}^h = \sum_{i=0}^n w_{ji}^h x_{pi}$$



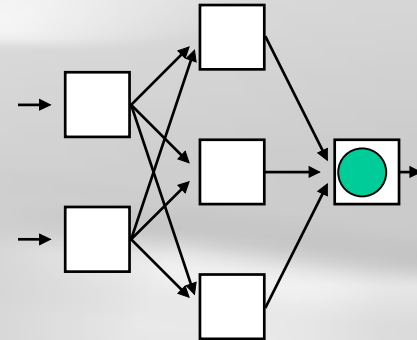
- obliczenie wartości wyjściowej y_{pj}^h dla każdego neuronu warstwy ukrytej

$$y_{pj}^h = f_j^h(u_{pj}^h)$$



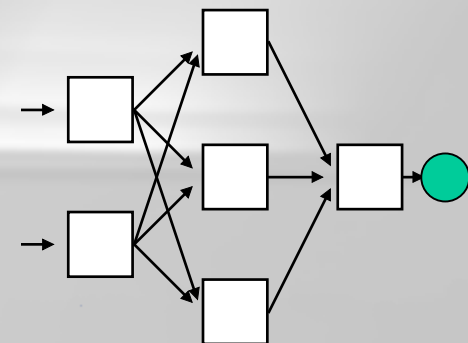
- obliczenie dla każdego neuronu warstwy wyjściowej wartości u_{pk}^{wyj} :

$$u_{pk}^{wyj} = \sum_{j=0}^I w_{kj}^{wyj} y_{pj}^h$$



- obliczenie wartości wyjściowej y_{pk}^{wyj} dla każdego neuronu warstwy wyjściowej

$$y_{pk}^{wyj} = f_k^{wyj}(u_{pk}^{wyj})$$



- ✓ Po wykonaniu wszystkich czynności wchodzących w skład wszystkich tych etapów sieć ustala swój sygnał wyjściowy y_{pk}^{wyj} .
- ✓ Sygnał ten może być prawidłowy lub nie, ale w tym właśnie celu mamy **proces uczenia**, żeby **rzeczywiste** działanie sieci dopasować do działania **pożądanego**.
- ✓ W prawidłowo działającej sieci wektor wartości wyjściowych y_p powinien być identyczny lub bardzo zbliżony do wektora wartości oczekiwanych d_p .

Określenie funkcji celu

- ✓ Podczas uczenia wagi są tak modyfikowane, aby błąd popełniany przez sieć malał, dlatego też konieczne jest zdefiniowanie funkcji celu a następnie jej minimalizacja.
- ✓ Najczęściej jako funkcję celu przyjmuje się błąd średniokwadratowy:

$$E = \frac{1}{2} \sum_{p=1}^q \sum_{j=1}^m \delta_{pj}^2$$

gdzie: q - liczba wektorów uczących;
 m - ilość neuronów w warstwie wyjściowej sieci.

Inne funkcje celu

W zależności od zastosowania, możemy definiować inne funkcje celu:

$$E = \sum_{p=1}^q \sum_{j=1}^m |\delta_{pj}|$$

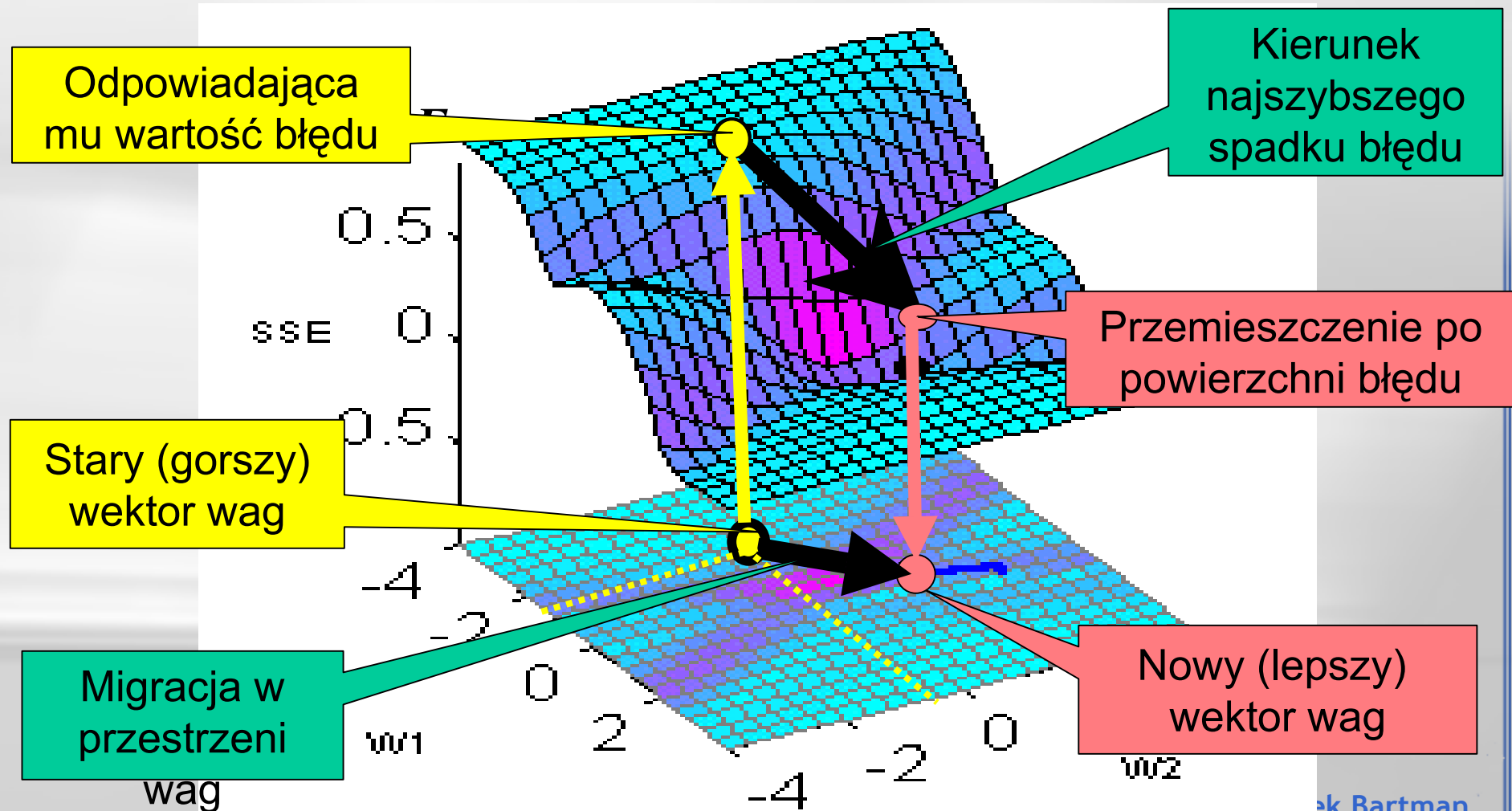
Funkcja ta zapewnia bardziej równomierny udział poszczególnych składników błędu w ogólnej definicji funkcji celu.

$$E = \sum_{p=1}^q \sum_{j=1}^m \delta_{pj}^{2K}$$

Funkcja ta minimalizuje największe odchylenia odpowiedzi od wielkości żądanej.

Idea

Metody gradientowe polegają na szukaniu kierunku spadku E i na takim zmienianiu wartości wag w_1, w_2 , żeby zmniejszać wartość funkcji błędu w kierunku jej **najszybszego spadku**



Reguła Delty

- ✓ W wyniku minimalizacji błędu średniokwadratowego uzyskujemy zależność opisującą modyfikację wag

$$\Delta \mathbf{w}_j = \eta \delta_j f'(\mathbf{w}_i^T \mathbf{x}) \mathbf{x}$$

jest ona znana jako **reguła Delty**

gdzie: f' - pochodna funkcji aktywacji;
 η - współczynnik uczenia.

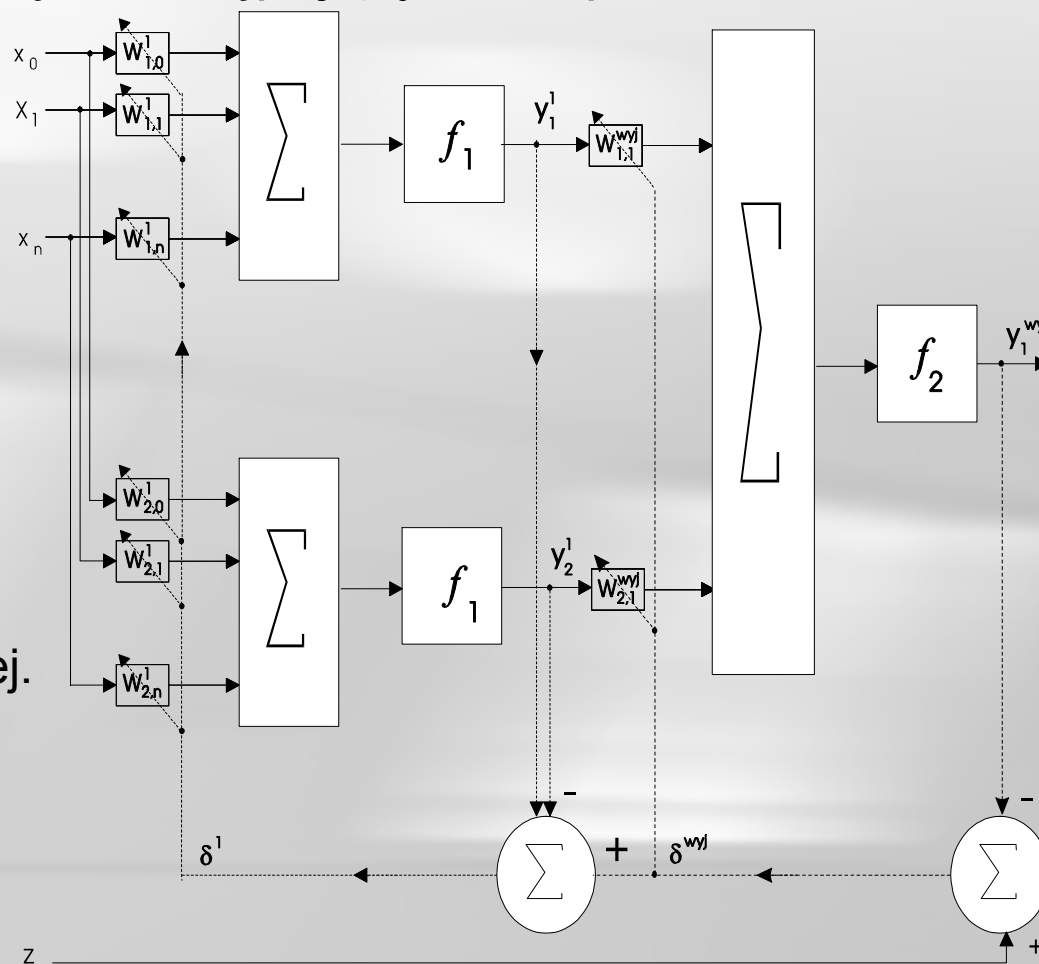
- ✓ Reguła Deltę jest najpopularniejszą metodą uczenia nadzorowanego.
- ✓ Praktycznie uczenie realizowane jest w następujący sposób:
 - na wejście sieci podawany jest sygnał wejściowy,
 - w oparciu o posiadaną wiedzę (posiadane wektory wag) neurony generują swoje odpowiedzi,
 - dla każdego neuronu wyznaczany jest błąd, jaki on popełnił, jako różnica pomiędzy odpowiedzią oczekiwaną a odpowiedzią udzieloną,
 - na podstawie wyznaczonego błędu obliczana jest korekta wektora wag neuronu.
- ✓ Regułę Deltę można stosować tylko w sieciach złożonych z neuronów o różniczkowalnych funkcjach aktywacji.

- ✓ Reguła Delty jest bardzo prosta do bezpośredniego zastosowania tylko wówczas, gdy **znana jest oczekiwana odpowiedź dla każdej warstwy sieci**, niestety, sytuacja taka występuje tylko w sieciach jednowarstwowych.
- ✓ Dla sieci wielowarstwowych najczęściej oczekiwane odpowiedzi dla **warstw pośrednich nie są znane** nie można więc wyznaczyć błędów popełnianych przez warstwę wejściową i warstwy ukryte, a jak nie jest znany błąd, to nie można na podstawie podanej reguły wyznaczyć pożądanej korekty wag dla tych warstw.
- ✓ Znana jest jednak oczekiwana odpowiedź dla warstwy wyjściowej i dla tej warstwy można wyznaczyć popełniany błąd, teraz wystarczy tylko rzutować go na warstwy poprzednie, aż do warstwy wejściowej.
- ✓ Metoda, która nakazuje takie postępowanie, nosi nazwę metody **wstecznej propagacji błędu (back propagation)** i bezsprzecznie jest to najczęściej stosowana metoda uczenia z nauczycielem sieci wielowarstwowych.

Metoda wstecznej propagacji błędów (backpropagation)

- ✓ Cykl uczenia metodą wstecznej propagacji błędów (backpropagation) składa się z następujących etapów:

1. Wyznaczenie odpowiedzi neuronów warstwy wyjściowej oraz warstw ukrytych na zadany sygnał wejściowy.
2. Wyznaczenie błęd popełnianego przez neurony znajdujące się w warstwie wyjściowej i przesłanie go w kierunku warstwy wejściowej.
3. Adaptacja wag.



- ✓ Realizacja pierwszego etapu algorytmu uczenia – wyznaczenie odpowiedzi neuronów - nie sprawia żadnych trudności.
- ✓ Również adaptacja wag występująca jako trzeci etap algorytmu jest możliwa do realizacji ; ale wymaga to znajomości błędów popełnianych przez wszystkie neurony - konieczne jest więc wykonanie p.2.
- ✓ Wsteczną propagację błędu najłatwiej jest przedstawić przy pomocy sieci zawierającej jedną warstwę ukrytą (przypadek ten zawsze daje się rozszerzyć na sieć o dowolnej liczbie warstw ukrytych). Można więc przeanalizować sieć dwuwarstwową o następującej budowie:
 - n - liczba neuronów wejściowych,
 - l - ilość neuronów w warstwie ukrytej
 - m - liczba neuronów wyjściowych,
 - q - długość ciągu uczącego

- ✓ Dla j - tego neuronu warstwy pierwszej wartość generowanej odpowiedzi można wyznaczyć z zależności:

$$y_j^1 = f(u_j^1) = f\left(\sum_{i=1}^n w_{ji}^1 x_i^{wej}\right)$$

- ✓ Ponieważ sygnał wyjściowy z tej warstwy jest sygnałem wejściowym dla warstwy następnej (warstwy wyjściowej), k - ty neuron tej warstwy wygeneruje odpowiedź:

$$y_k^{wyj} = f(u_k^{wyj}) = f\left(\sum_{j=1}^l w_{kj}^{wyj} y_j^1\right)$$

$$y_k^{wyj} = f\left(\underbrace{\sum_{j=1}^l w_{kj}^{wyj} f\left(\sum_{i=1}^n w_{ji}^1 x_i^{wej}\right)}_{u_k^{wyj}}\right)$$

- ✓ W przypadku wieloelementowego ciągu uczącego błąd średnio-kwadratowy można zapisać w postaci:

$$E = \frac{1}{2} \sum_{p=1}^q \sum_{k=1}^m (d_{pk} - y_{pk})^2$$

- ✓ W przypadku jednoelementowego ciągu uczącego w postaci:

$$E = \frac{1}{2} \sum_{k=1}^m (d_{pk} - y_{pk})^2$$

- ✓ Uaktualnianie wag uczonych neuronów może odbywać się:
 - ➡ po prezentacji całego ciągu uczącego - wówczas funkcję celu definiuje się w postaci (wzór pierwszy)
 - ➡ po prezentacji każdego elementu - wówczas funkcję celu definiuje się w postaci (wzór drugi).
- ✓ Przyjmijmy, że aktualizacja wag odbywa się każdorazowo po podaniu elementu ciągu uczącego.

dla warstwy wyjściowej sieci:

$$E = \frac{1}{2} \sum_{k=1}^m (d_k - y_k^{wyj})^2 \rightarrow \frac{\partial E}{\partial y_k^{wyj}} = -(d_k - y_k^{wyj})$$

$$y_k^{wyj} = f(u_k^{wyj}) \rightarrow \frac{dy_k^{wyj}}{du_k^{wyj}} = \frac{df}{du_k^{wyj}}$$

$$y_k^{wyj} = f \left(\underbrace{\sum_{j=1}^I w_{kj}^{wyj} f \left(\sum_{i=1}^n w_{ji}^1 x_i^{wej} \right)}_{u_k^{wyj}} \right)$$

$$\frac{\partial u_k^{wyj}}{\partial w_{kj}^{wyj}} = f \left(\sum_{i=1}^n w_{ji}^1 x_i^{wej} \right) = y_j^i$$

$$\Delta w_{kj} = -\eta \frac{\partial E}{\partial w_{kj}}$$



własności
pochodnych

$$\Delta w_{kj} = -\eta \frac{\partial E}{\partial y_k^{wyj}} \frac{dy_k^{wyj}}{du_k^{wyj}} \frac{\partial u_k^{wyj}}{\partial w_{kj}^{wyj}}$$

Architektura sieci $\rightarrow y_j^i = x_j^{wyj}$

dla warstwy wyjściowej sieci:

$$E = \frac{1}{2} \sum_{k=1}^m (d_k - y_k^{wyj})^2 \rightarrow \frac{\partial E}{\partial y_k^{wyj}} = -(d_k - y_k^{wyj})$$

$$y_k^{wyj} = f(u_k^{wyj}) \rightarrow \frac{dy_k^{wyj}}{du_k^{wyj}} = \frac{df}{du_k^{wyj}}$$

$$y_k^{wyj} = f \left(\underbrace{\sum_{j=1}^I w_{kj}^{wyj} f \left(\sum_{i=1}^n w_{ji}^1 x_i^{wej} \right)}_{u_k^{wyj}} \right)$$

$$\frac{\partial u_k^{wyj}}{\partial w_{kj}^{wyj}} = f \left(\sum_{i=1}^n w_{ji}^1 x_i^{wej} \right) = y_j^i$$

Architektura sieci $\rightarrow y_j^i = x_j^{wyj}$

$$\Delta w_{kj} = -\eta \frac{\partial E}{\partial w_{kj}}$$

↓
własności
pochodnych

$$\Delta w_{kj} = -\eta \frac{\partial E}{\partial y_k^{wyj}} \frac{dy_k^{wyj}}{du_k^{wyj}} \frac{\partial u_k^{wyj}}{\partial w_{kj}^{wyj}}$$

$$\Delta w_{kj}^{wyj} = \eta (d_k - y_k^{wyj}) \frac{df}{du_k^{wyj}} x_j^{wyj}$$

$$\delta_k^{wyj} = d_k - y_k^{wyj}$$

dla warstwy ukrytej:

$$E = \frac{1}{2} \sum_{k=1}^m (d_k - y_k^{wyj})^2 \rightarrow \frac{\partial E}{\partial y_k^{wyj}} = -(d_k - y_k^{wyj})$$

$$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}}$$



własności
pochodnych

$$\frac{\partial y_k^{wyj}}{\partial u_j^1} = \frac{\partial f(\sum (w_{kj} * f(u_j^1)))}{\partial u_j^1} = \frac{df(u_k^{wyj})}{du_k^{wyj}} w_{kj} \frac{df(u_j^1)}{du_j^1}$$

$$\frac{\partial E}{\partial w_{ji}} = \sum_{k=1}^m \frac{\partial E}{\partial y_k^{wyj}} \frac{\partial y_k^{wyj}}{\partial u_j^1} \frac{\partial u_j^1}{\partial w_{ji}}$$

$$\frac{\partial u_j^1}{\partial w_{ji}} = x_i^{wej}$$

dla warstwy ukrytej:

$$E = \frac{1}{2} \sum_{k=1}^m (d_k - y_k^{wyj})^2 \rightarrow \frac{\partial E}{\partial y_k^{wyj}} = -(d_k - y_k^{wyj})$$

$$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}}$$



własności
pochodnych

$$\frac{\partial y_k^{wyj}}{\partial u_j^1} = \frac{\partial f(\sum (w_{kj} * f(u_j^1)))}{\partial u_j^1} = \frac{d f(u_k^{wyj})}{d u_k^{wyj}} w_{kj} \frac{d f(u_j^1)}{d u_j^1}$$

$$\frac{\partial E}{\partial w_{ji}} = \sum_{k=1}^m \frac{\partial E}{\partial y_k^{wyj}} \frac{\partial y_k^{wyj}}{\partial u_j^1} \frac{\partial u_j^1}{\partial w_{ji}}$$

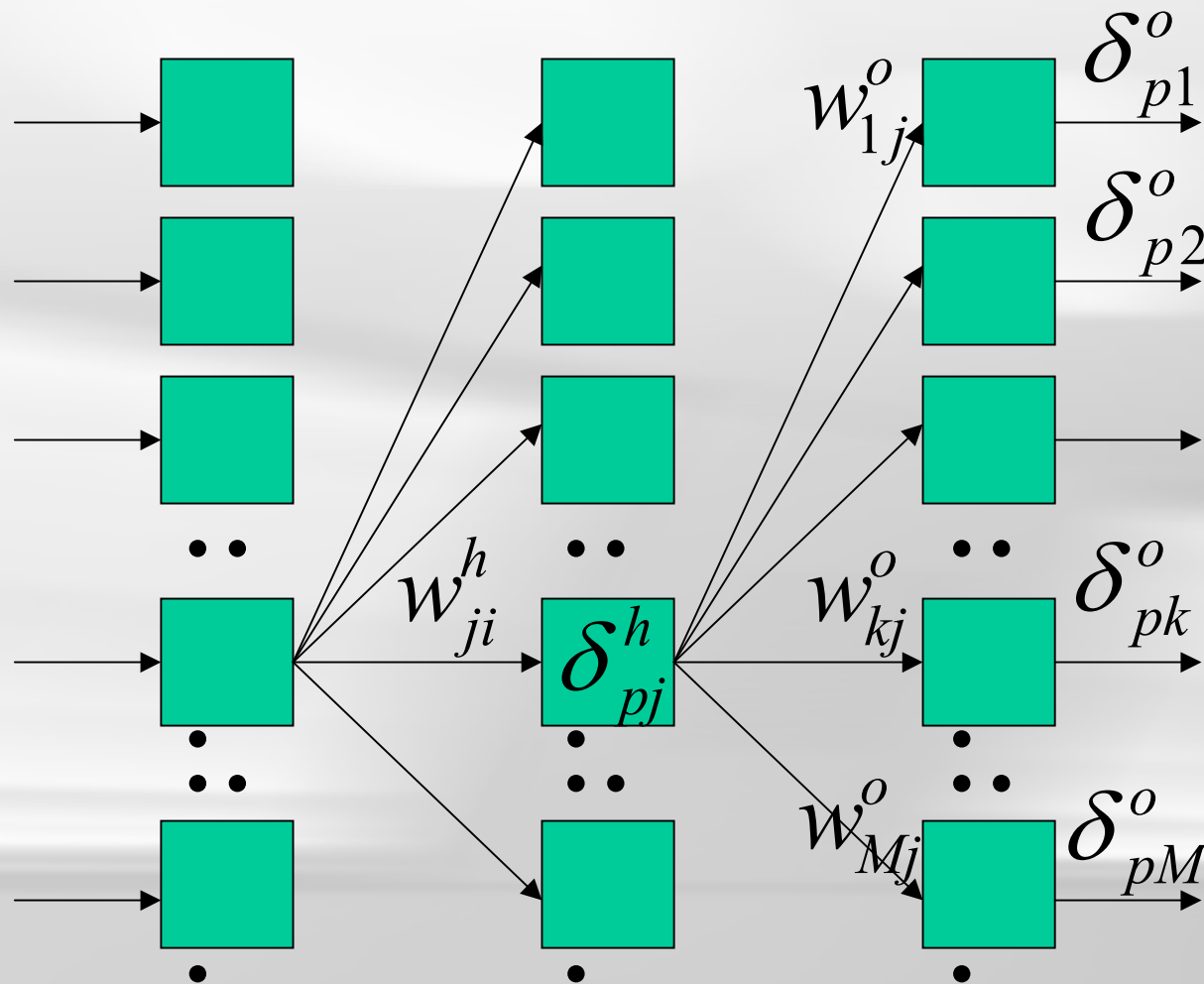
Równanie pozwala wyznaczyć błąd dla dowolnego neuronu warstwy ukrytej w funkcji błędów neuronów, które on pobudza. Innymi słowy umożliwia przenoszenie błędu wstecz (od warstwy wyjściowej ku wejściowej).

$$\frac{\partial u_j^1}{\partial w_{ji}} = x_i^{wej}$$

$$\Delta w_{ji}^1 = \eta \sum_{k=1}^m (d_k - y_k^{wyj}) \frac{d f(u_k^{wyj})}{d u_k^{wyj}} w_{kj} \frac{d f(u_j^1)}{d u_j^1} x_i^{wej}$$

$$\delta_i^1 = \sum_{k=1}^m (d_k - y_k^{wyj}) \frac{d f(u_j^1)}{d u_j^1} w_{kj} \overset{\delta_k^{wyj} = d_k - y_k^{wyj}}{=} \sum_{k=1}^m \delta_k^{wyj} \frac{d f(u_j^1)}{d u_j^1} w_{kj}$$

Poprawić i dodać wzór



Algorytm wstecznej propagacji błędu

1. Wygeneruj losowo wektory wag.
2. Podaj wybrany wzorzec na wejście sieci.
3. Wyznacz odpowiedzi wszystkich neuronów wyjściowych sieci:
4. Oblicz błędy wszystkich neuronów warstwy wyjściowej:

$$y_k^{wyj} = f\left(\sum_{j=1}^I w_{kj}^{wyj} y_j^{wyj-1}\right)$$

$$\delta_k^{wyj} = z_k - y_k^{wyj}$$

5. Oblicz błędy w warstwach ukrytych (pamiętając, że, aby wyznaczyć błąd w warstwie $h - 1$, konieczna jest znajomość błędu w warstwie po niej następującej - h):

$$\delta_j^{h-1} = \frac{df(u_j^{h-1})}{du_j^{h-1}} \sum_{k=1}^I \delta_k^h w_{kj}^h$$

6. Zmodyfikuj wagi wg zależności:
7. Wróć do punktu 2.

$$w_{ji}^{h-1} = w_{ji}^{h-1} + \eta \delta_j^{h-1} y_i^{h-1}$$

Wady backpropagation

- ✓ Nie można zagwarantować, iż proces uczenia doprowadzi do odnalezienia minimum globalnego funkcji miary błędu - często zdarza, że odnalezione zostaje minimum lokalne,
 - Wybranie niewłaściwego punktu startowego czyli niewłaściwy dobór wartości początkowych wag oraz nieodpowiedniej drogi może spowodować wejście w minimum lokalne, którego algorytm nie będzie w stanie opuścić.
- ✓ Funkcja miary błędu jest funkcją wielokrotnie symetryczną w wielowymiarowej przestrzeni wag, co powoduje występowanie wielu minimów globalnych i lokalnych.
- ✓ Klasyczna metoda wstecznej propagacji błędów wymaga dużej liczby iteracji by osiągnąć zbieżność oraz jest wrażliwa na występowanie minimów lokalnych.
- ✓ Podstawowy algorytm BP może się okazać **zbyt wolny**, jeżeli przyjmie się **za mały współczynnik uczenia**, z kolei **zbyt duża wartość współczynnika** grozi wystąpieniem oscylacji.